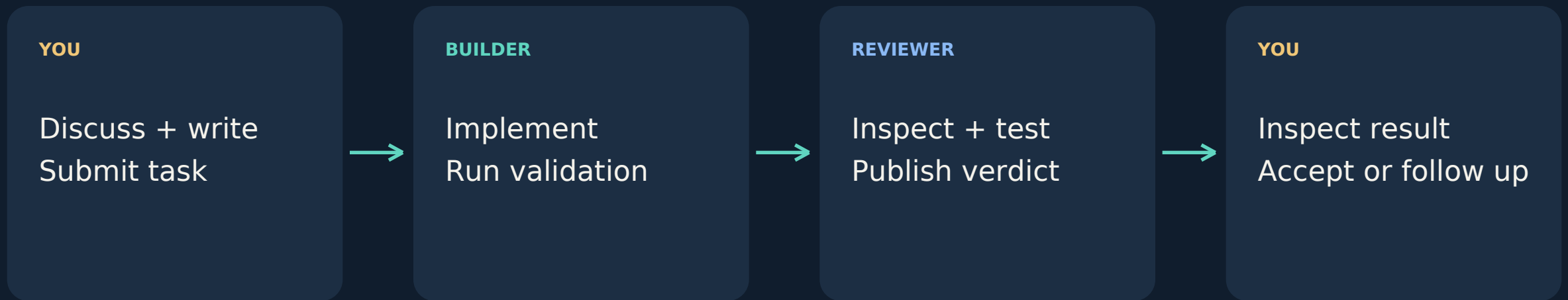


Commit an instruction. Start a supervised cycle.

You define the work. Claude builds it. Codex checks it. You decide what happens next.

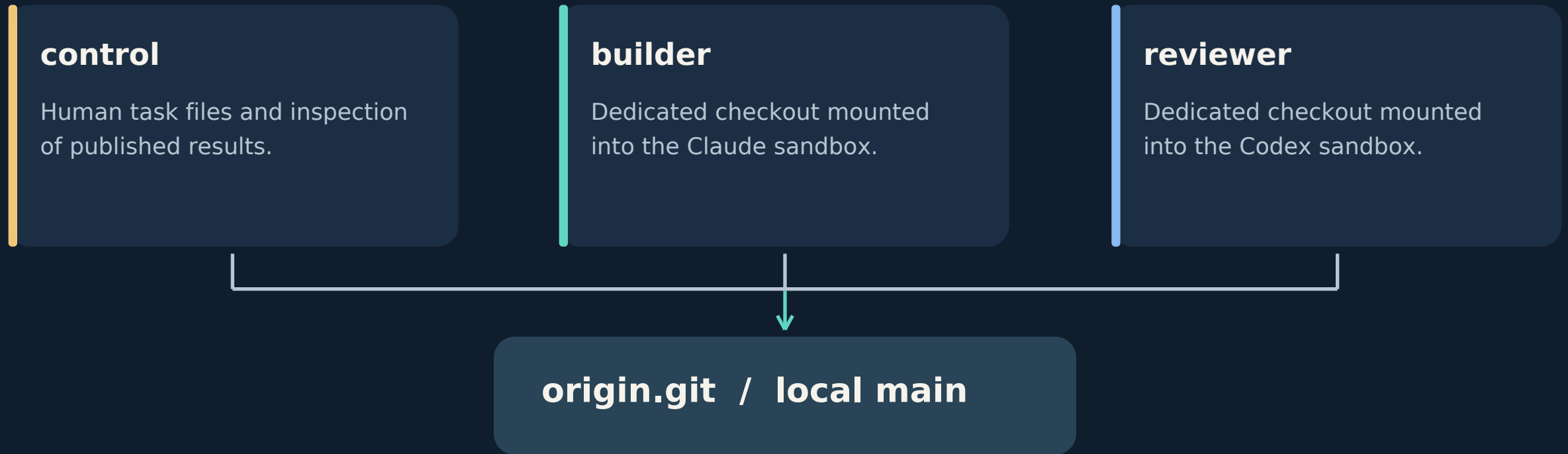


The committed instruction is the authorization record.

A saved draft is a proposal. Submission is your signoff.

One project. Three working copies.

Everything below is inside agentic-ams-elarning-automatisation on the Windows host.



logs + state

Execution evidence and processed-event memory.

Azure is a separate destination

Local pushes do not publish to the external remote.

The dispatcher watches commits, not files.

The host polls local origin/main and routes commits using event metadata.

1 Instruction

instruction

Submit-Task.ps1 creates the task commit and pushes it locally.

2 Implementation

build-complete

Claude makes one commit. The host validates it and publishes it.

3 Review

review-complete

Codex makes one review commit. The host validates and publishes it.

Implementation reaches local main BEFORE review. A PASS is evidence for your decision; it is not an automatic merge or Azure publication.

A review ends with one of three verdicts.

Every follow-up stays supervised: draft, promote, edit, submit.

PASS

Inspect the report and changes.
Accept the work if satisfied.

CHANGES_REQUESTED

Inspect the proposed fixes. Decide
whether to authorize them.

DECISION_REQUIRED

Resolve the choices. Write the
decisions into requirements.



Committed draft → Promote → Edit + save → Submit

Promotion creates a new task ID; it does not start an agent. Submission is signoff. Keep parent-task links. Default limit: three follow-up rounds.

Two tabs. Both start in the control folder.

The tab name tells you its job; the directory tells you where commands run.

DISPATCHER TAB

Start Start-AgentPipeline.ps1 with the project sandbox names.

Watch build, review and publication messages.

Leave it running while a task is active. Stop with Ctrl+C when idle.

CONTROL TAB

Discuss and save a task. Inspect it before submission.

Run Submit-Task.ps1.

After review publication: pull, read the report and inspect the exact implementation diff.

New tab: set process execution policy and navigate to control. Use the companion operating notes for copy-ready commands.

Host orchestration. Isolated agent environments.

Windows PowerShell and the Linux sandbox are separate environments.

WINDOWS HOST

Dispatcher + Git synchronization
Local origin.git publication
Task submission + human review

CLAUDE SANDBOX

Builder checkout
Own installed dependencies
Own Python environment
Implementation + validation

CODEX SANDBOX

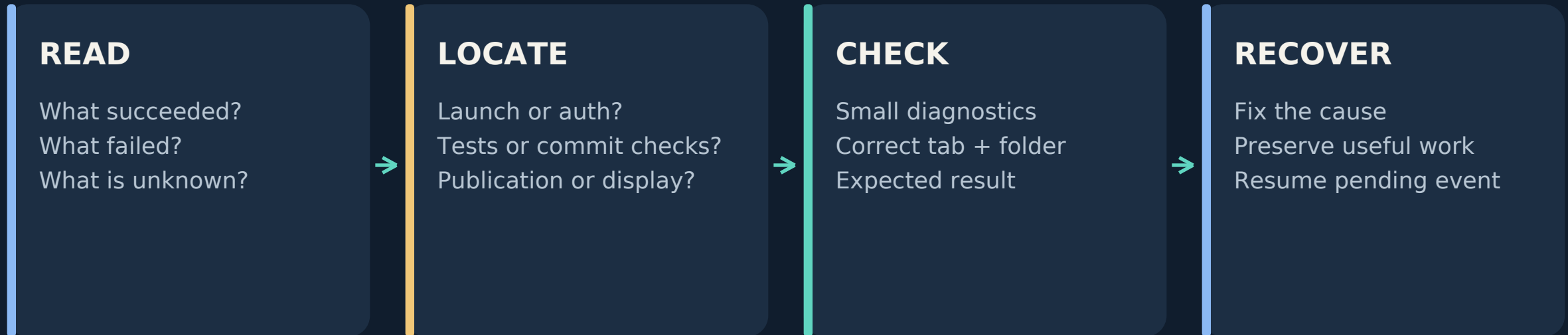
Reviewer checkout
Own installed dependencies
Own Python environment
Independent validation

`/home/agent/.venvs/ams-elearning`

Keep dependencies outside the checkout: synchronization resets and cleans the dedicated agent clones, including ignored files. Host Conda does not supply sandbox packages.

Find the failed stage before choosing a fix.

Keep the published work and processed state. Do not repeat unchanged retries.



A red status line is not proof of failure.

Check the exit code and publication message. A failed launch is not a negative review.

The workflow has completed a real feedback loop.

Historical evidence from TASK-0100 and TASK-0101. Recheck current state in a new chat.

TASK-0100

Build documentation
and task-authoring skill



REVIEW

Changes requested:
validator wording + links



TASK-0101

Human-submitted fixes
Independent review: PASS

27

offline Python tests passed
during the successful review

Ready for the next concrete task

Keep human signoff for follow-ups. Inspect every result. Add validation when new scope needs it. Publishing to Azure remains a separate decision.